

代码考古学

产品价值论证报告

这个产品是什么、为什么必须存在、凭什么值得被信任

报告性质	产品价值论证（Product Value Dossier）— 面向评委、合作方与技术尽调的完整论证
产品名称	代码考古学 / Code Archaeology
产品定位	信息丢失审计工具：量化一个此前从未被量化的指标——代码库的「知识可继承性」
线上入口	https://code-archaeology.pages.dev （实测 HTTP 200，内地直连可达）
产品形态	Web 应用 · HTTP API · 命令行（CLI）· 桌面客户端（macOS / Windows）
技术底座	Node.js ESM + Cloudflare Pages / Workers（边缘运行时）+ GitHub REST API
编写人	蔡奇均 Qijun Cai · 王一博 Yibo Wang
签发人	蔡奇均 Qijun Cai · 王一博 Yibo Wang
报告时间	2026 年 9 月 20 日 09:59（UTC+8）
数据基准	本报告全部数字于报告生成时刻实时采集（线上 API 实测 + 本地测试实跑 + 源码静态统计）

一句话定位（全篇论证的锚点）

我们不告诉你「为什么」，只告诉你「哪里已经没人能说清为什么」。

We don't tell you why — we tell you where nobody can explain why anymore.

这句话不是修辞，而是产品的能力边界声明：它承诺的每一件事都可被第三方复核，它不承诺的那件事（重建决策动机）恰恰是它拒绝越界的地方。

本报告共十一章。第二至四章回答「是什么、为什么做、怎么做」，第五章给出四重价值论证，第六章是本次生成时刻的完整实测记录，第十章主动披露已知边界——我们认为，一份只讲优势、不提边界的论证报告，本身就不值得被信任。

4 项 可被第三方复核的客观指标 (信噪比 / 含糊率 / 巴士因子 / 断层)	3 段 有依赖关系的流水线 (非并行投票, 后一步消费前一步)	100% 本次实测证据接地率 10 条引用全部命中真实提交	181/181 单元测试通过 0 失败 (本地实跑)
---	--	--	---

结论：这个产品存在的价值，不在于它能「读懂」代码，而在于它把一件过去只能靠口耳相传、且极易在人员流动中彻底消失的东西——「这段代码是怎么变成今天这样的」——变成了可测量、可复核、可随时间追踪的指标。

- 它作用的对象不是代码，是历史的沉积层。当前代码只呈现「最终结果」，而「试过什么、放弃了什么、为什么放弃」这类信息在代码里往往零痕迹——没有 TODO、没有注释、没有残留分支。它只存在于提交历史里。
- 它解决的是一类被系统性忽略的损失。机器人提交（依赖升级、锁文件）会淹没人类的真实决策；「fix」「update」这类提交信息会让改动意图永久蒸发；先进后出的改动会让代码库丢失自己的演化证据。三者叠加的后果不是「代码质量差」，而是这个项目还能不能被交接。
- 它的可信度来自设计，而不是来自自信。系统里所有推断都必须引用真实存在的 commit SHA，而「引用了不存在的 SHA」这件事由代码通过集合运算判定，不经过任何模型——这是可证明的编造，不是主观评价。本次实测：6 条结论 / 10 条引用 / 0 条编造。
- 它证明过自己是错的。产品核心指标曾在主力演示仓库上产生假阳性，团队逐条核对 GitHub 真实 diff 后将其修复，并把「复核未通过」的判定一并公开展示。一个敢于在交付物里暴露自身误判的系统，其正向结论才具备了可信度基础。

2.1 第一对象：提交历史的沉积层，而不是代码本身

考古学的基本前提是：地层中有两类东西 —— 沉积物（自然堆积）与文物（人类活动的产物）。一个不区分二者的挖掘者，挖出来的全是泥沙。代码仓库完全同构：机器人提交与生成物是沉积物，人类带着判断做出的改动才是文物。产品的第一件事，就是把后者从前者里分离出来。

同一件事	读「当前代码」	读「提交历史」	本产品如何呈现
这段代码为什么长这样	只能看到结果	能看到改动顺序与取舍	时间轴 + 文件变更序列，每条附 commit SHA
试过什么方案、为什么放弃	往往零痕迹	可被重建（先进后出）	「被放弃的尝试」：内容重合率核实
谁在决策、什么时候改变方向	看不见	能看到节奏与断层	提交节奏、最长沉寂期、沉寂后首个提交
有多少改动其实不是人做的	完全看不见	可精确计数	信噪比：机器人提交占比、噪音文件排行
改动意图还剩下多少	看不见	可度量	提交信息质量：含糊提交率（风险分第一因子）

2.2 第二对象：三类真实使用者

使用者	他们面对的具体处境	产品交付给他们的东西
接手代码的人	拿到一个别人写了很久的项目，没人可以问，历史又读不完	一份「哪些地方已经没人能说清」的清单 + 交接前该问的 3 个问题
做技术尽调的人	要在有限时间内判断一个代码库的真实健康度与知识集中风险	可复核的客观指标：风险分、巴士因子、知识集中文件、方向摇摆证据
项目团队自身	想知道自己的提交习惯正在制造多少未来的负债	一台可重复运行的仪表：三个月后再跑一次，看指标有没有变好

2.3 对象边界：这个产品不作用于什么

明确边界与明确能力同等重要。产品的取数与判断全部限定在 Git 提交史 之内：它不读 PR 描述、不读 issue、不读代码注释、不读团队聊天记录。这不是能力缺失，而是刻意的取舍 —— 少一类证据源，就少一类无法自证的空间。在这个前提下，所有结论都建立在「可从 GitHub 原始接口逐条回查」的对象之上。

3.1 三类信息，读代码永远看不到

① 沉积物淹没文物 机器人提交会系统性地稀释人类的真实决策密度。本次实测：`chalk/chalk` 最近 40 次提交中 33 次含人类决策（83%），而 `ody-cai/oa-system` 为 29 次（73%）。在依赖自动化的仓库里这个比例会倒转 —— 项目文档记录的另一抽样中，`axios/axios` 的 40 次提交有 20 次来自 `dependabot`，高达 50%。不做信噪分离，报告的第一页就会是依赖升级清单。

② 意图随提交信息蒸发 「fix」「update」「修改」让后来者永远无法还原改动原因。实测含糊提交率：`chalk/chalk` 36%，`ody-cai/oa-system` 14%，平均提交信息长度仅 20 – 22 个字符。这类损失是不可逆的 —— 它不体现在任何一次代码审查里，只在两年后某人试图改动这段代码时才爆发。

③ 先进后出，代码里零痕迹 一个功能被加上、又在短期内被整体拆除时，代码库中不会留下任何证据。没有注释、没有 TODO、没有残留分支 ——

下一个接手者会再次尝试同一个方向。这是唯一只能从历史中获取、且当前代码中绝对不存在的信息。

问题的收束

三类信息丢失叠加后，暴露出来的不是「这个项目好不好」，而是「它还能不能被交接」。

—— 「好不好」是主观评价，无法验证；「能不能交接」是客观状态，可以被量化、被追踪、被比较。产品选择回答后者。

3.2 为什么现有工具覆盖不到

业界已有相当成熟的「行为码分析」（behavioral code analysis）工具族 —— hotspot 分析、bus factor、knowledge map、团队效能度量。它们与我们的重合部分，我们承认：本产品在热点文件、作者分布、巴士因子这几项上没有本质优势，部分是同类方法的重新实现。

差异只有一条，而且很窄：「被放弃的尝试」—— 指那些在当前代码中已不存在、只能从历史中挖出的信息。由于这条护城河很窄，团队选择把它做扎实而不是做宽：引入内容级核实（按内容判定，而非按行数猜测）、排除整仓回滚的连带删除、并对不可核实的条目拒不下判断。

产品的完整链路是串行有依赖的三段。这不是为了凑技术复杂度 —— 每一段都在为下一段削减不确定性，最后一段则负责审计前两段的诚实度。

STAGE 01	信噪分离 纯计算 · 不调用任何模型	机器人提交识别（GitHub Bot 标记 + 自建 bot 命名规则）；生成物/锁文件/二进制/构建产物识别；只保留「人类提交 且 至少改动一个非生成物文件」的提交进入下游。输出信噪比。
STAGE 02	多模型分工 3 家供应商 · 有依赖的流水线	叙事者（gpt-4o）把 diff 序列变成事件过程 假设者（deepseek-v4-pro）据此推断意图，二者并行；审稿人（gemini-2.5-flash）拿到代码层校验结果后回头复核并点名质疑前两者的结论。
STAGE 03	证据接地校验 SHA 集合运算 · 不依赖模型自评	把「模型引用了不存在的提交」判定为可证明的编造。这一步 100% 由程序完成，不经过任何模型，因此不存在「判官本身也幻觉」的失效模式。

4.1 STAGE 01 · 信噪分离：先筛地层，再挖文物

这一步不调用任何模型，是纯函数式的规则判定，因此可以被单独测试、单独扩展、也可以被其他分析工具复用。判定分两个维度：

- 谁是机器人：authorType / committerType 为 Bot 的直接判定；此外用一套命名规则覆盖大量自建机器人（dependabot、renovate、github-actions、[bot] 后缀等十余种形态）。
- 什么是沉积物：锁文件（package-lock.json、yarn.lock、go.sum 等）、构建产物目录、压缩/映射文件、变更日志与贡献者名单、测试夹具与示例数据、以及全部二进制与媒体文件。

为什么这一步必须排在第一位

如果不先剥离沉积物，后续所有指标 —— 风险分、热点文件、被放弃的尝试 —— 都会被依赖升级的噪音稀释到失去意义。因此系统对外交付的第一个数字是信噪比，而不是结论。这是一个刻意的产品选择：先让使用者知道这份报告的证据基础有多厚，再让他读结论。

4.2 STAGE 02 · 多模型分工：为什么不是「并行投票」

三个模型不是各答一遍然后投票。它们之间存在真实的依赖关系：叙事者把零散的 diff 变成有时间顺序的事件过程；假设者基于这个过程推断「当初想做什么、为什么放弃」；审稿人拿到前两者的产出以及代码层的校验结果，再回头提出质疑。

后一步消费前一步的产出 —— 这才叫工作流。单模型换 prompt 无法复现这个闭环，因为审稿人的一句关键质疑是「你这条结论引用了不存在的提交」，而这种话只有在它能访问独立于模型的校验结果时才说得出来。叙事者与假设者之间没有依赖，因此并行执行；本次实测完整链路耗时 29.8 秒。



4.3 STAGE 03 · 证据接地校验：幻觉检测不靠模型，靠代码

业界常见做法是「让另一个模型检查前一个模型有没有胡说」。这个方案有一个无法修补的缺陷：判官本身也会幻觉，而且它无法查证。

本产品的做法是：模型产出的每一条结论都必须引用具体的 commit SHA，而这些 SHA 是系统自己从 GitHub 真实拉回来的。于是「引用了一个不存在的 SHA」这件事，可以被一次集合运算直接判定 —— 这是可证明的编造，不是主观判断。这一步不经过任何模型。

校验状态	判定条件	含义
grounded（已接地）	给出了引用，且全部命中真实提交	结论有据可查，可被第三方复核
ungrounded（无证据）	没有给出任何引用	结论未被支持，不计入可信输出
fabricated（编造）	给出的引用中存在不存在的提交	可证明的幻觉，直接暴露在报告里

4.4 把「说不清」变成「可测量」：指标定义

产品价值的技术前提，是每一个结论都对应一个有明确定义、可由原始数据重算的指标。这是「凭什么可信」的第一层答案。

指标	计算方式	回答什么问题
信噪比	机器人提交数 / 分析提交总数	这份报告的证据基础有多厚
含糊提交率	含糊信息提交数 / 非合并的人类提交数	改动意图还剩多少可以被还原
被放弃的尝试	某次净增 >= 阈值，随后 <=45 天内净删 >= 阈值 60%，且拆除内容与新增内容重合 >= 50%	哪些方向被走过后又丢掉了
原地打转	改动总量 >= 40 且 净变化 <= 改动总量 x 30%	哪些文件在被反复推翻
巴士因子（知识集中）	只被单一作者改动过、且被改动 >= 2 次的文件	哪些文件只有一个人懂
最长沉寂期	相邻人类提交之间的最大时间间隔	哪里出现过断裂，以及重启后的第一个动作
风险分	五个因子加权求和（25/25/20/15/15），上限 100	整体知识可继承性的综合刻度

说明：风险分是团队设定的加权模型，属于设计选择而非可核验事实，本报告在使用该数字时始终同时给出全部五个因子的原始值，供使用者自行重新加权。其余各项均可由 GitHub 原始响应直接重算。

「有价值」如果只靠形容词，就无法被检验。因此我们把它拆成四条可被逐一验证的论证。

论证一 · 它把不可测量变成了可测量

在它之前，「这段代码还有没有人说得清」是一个只能靠资深工程师个人感觉回答的问题，无法比较、无法追踪、无法写进交接文档。产品的核心贡献是给出了一组有明确定义、可重算的替代指标。一个指标只要能被重算，就能被讨论；能被讨论，才能被改进。

论证二 · 每一条结论都可回溯到原始证据

系统输出的每一条推断都带 commit SHA，使用者可以逐条回查。更进一步：系统会主动统计自己的接地率，并把编造的条目单列出来。一份不给出证据来源、也不统计自身错误率的分析报告，无论措辞多么肯定，都不具备参考价值。

论证三 · 结论可在第三方机器上完整复现

产品没有预置数据集，没有合成 fixture 出现在演示路径上。任何人都可以指定任意一个公开仓库，当场得到结果。附录 A 给出了完整的复现命令——复现路径不依赖团队的任何私有环境。

论证四 · 它验证过自己是错的（可信度的最强证明）

这是全部论证中最关键的一条。产品最核心的指标——「被放弃的尝试」——曾经在一次针对主力演示仓库的复核中被发现是假阳性。最初的判据只看净删行数，不看内容，于是：

仓库 / 文件	提交配对	内容重合率	真相
oa-system (邮件发送路由)	b86cce0 (+58) -> 7c78ec0 (-58) 间隔 52 分钟	100%	真阳性：方案确被放弃
chalk/chalk (test/chalk.js)	ff549c5 (+20) -> 8a94e0e (-36)	31%	假阳性
oa-system (AGENTS.md)	d8ca09f (+43) -> 109fd2c (-119)	28%	假阳性（整仓回滚连带）
chalk/chalk (source/index.js)	4c304dd (+19) -> 5729845 (-35)	14%	假阳性：实为性能重写

被误判的那次：5729845 的提交信息是 **Improve performance**，它是 4c304dd 的直接子提交，间隔仅 1 小时 20 分。它重写了实现，而不是放弃了功能——核对当前 main 分支，「扩展下划线样式」仍是该项目完整保留的核心特性。

修复方式与修复结果

新增独立模块，把判定从「按行数猜」升级为「按内容判」：内容核实（拆除侧删掉的行在新增侧出现的比例 $\geq 50\%$ 才算放弃）、排除整仓回滚（连带删除单列，不归因为文件级放弃）、不可核实则不下判断。

阈值 50% 直接取自上面这张实测分布——真阳性 100%、假阳性 14/28/31%，两簇之间隔着约 50 个百分点、完全不重叠，因此阈值是测量结果，不是估计值。

效果：chalk 风险分由 63（高）降至 55（中），降幅全部来自假阳性被剔除；而 oa-system 的真实案例（间隔 52 分钟）完整保留。

为什么这条构成价值论证而不是减分项：一个从未被证伪过的指标，我们对它的置信度只能来自直觉；一个被证伪并修复过的指标，它的准确度是被实际测量过的。修复过程同时暴露了一个更深的设计原则——把「复核未通过」的判定一并展示在页面上。只展示「发现了什么」而不展示「否定了什么」的报告，无法被复核，因此也就不具备被信任的资格。

以下全部数据于报告生成时刻实时采集，采集方式与结果一并列出，不含任何历史引用或估算值。

6.1 线上服务可用性

探测项	结果	说明
GET /api/health	HTTP 200 · 0.76 s	模型层已配置（provider=openai），三个模型就位
GET /（前端）	HTTP 200 · 0.75 s	内地网络直连可达（未使用代理）
POST /api/mine（oa-system，40 提交）	HTTP 200 · 3.40 s	返回 62.9 KB 完整报告
POST /api/mine（chalk，40 提交）	HTTP 200 · 3.09 s	纯计算链路，不调用模型
POST /api/excavate（oa-system，40 提交）	HTTP 200 · 29.80 s	三模型完整流水线，无降级

线上入口：<https://code-archaeology.pages.dev> · 健康检查返回的三模型分工：叙事者 gpt-4o、假设者 deepseek-v4-pro、审稿人 gemini-2.5-flash（3 家不同供应商）。

6.2 两个真实仓库的挖掘结果（对照）

指标	ody-cai/oa-system	chalk/chalk	解读
分析提交数	40	40	窗口一致，可横向比较
机器人提交	0（0%）	0（0%）	两个仓库此窗口内均无自动化提交
含人类决策的提交	29（73%）	33（83%）	有效考古线索
含糊提交率	14%	36%	chalk 的提交信息质量明显更低
平均提交信息长度	22 字符	20 字符	两者都偏短，意图留存不足
被放弃的尝试（已核实）	4 处	0 处	本产品的核心差异化指标
· 复核未通过（已否定）	3 处	2 处	假阳性被单列，不混入结论
· 整仓回滚连带删除	10 处	0 处	原因不同，不归因为文件级放弃
原地打转的文件	9 个	3 个	改动量大但净变化小
知识集中文件（巴士因子）	8 个	7 个	只被一个作者改动过
最长沉寂期	100 天	225 天	中断后重启的位置往往伴随转向
风险分	71（高）	55（中）	五个因子加权结果，原始值见下
API 调用次数	42 次	42 次	配额余量充足（实测余量 >4800）

oa-system 风险分 71（高）的因子拆解：提交信息质量 18/25（4 条无信息量 / 29 条） · 原地打转 25/25（9 个文件反复推翻） · 被放弃的尝试 16/20（4 处已核实） · 显式回滚 0/15（0 次） · 最长沉寂期 13/15（100 天）。注意其中最后两项的对比 —— 系统没有因为「没有回滚」就给出一个笼统的低分，而是把高分明确归因到具体因子。

6.3 完整多模型流水线输出（oa-system）

环节	实测产出
叙事层	2 条转折点（快速功能开发 -> 大规模功能回退），标题「从快速迭代到功能回退的初期开发历程」
假设层	4 条假设，每条均带「尝试了什么 / 为什么放弃 / 常见模式 / 置信度」四项；置信度分布：1 条高、3 条中
审稿层	裁决「这份代码的历史表面清晰、实则内部混乱」；给出 3 项风险归因、4 条交接建议、2 条质疑、3 个交接必问问题
证据接地	结论 6 条 / 引用 10 条 / 命中 10 条 / 编造 0 条 -> 接地率 100%
降级情况	degraded = false，无任何环节降级

6.4 真实案例：52 分钟内被整体拆除的功能

b86cce0	2026-05-29 12:51	"feat: 集成邮件发送功能 (Resend) "
	src/app/api/email/send/route.ts	+58 行
	间隔 52 分钟	
7c78ec0	2026-05-29 13:43	"fix: 清理不需要的邮件相关代码和依赖"
	src/app/api/email/send/route.ts	-58 行
		内容重合率 100%

这是只有读历史才能得到的结论。当前代码里没有任何证据表明这个功能存在过——没有注释、没有残留引用、没有废弃标记。系统的假设层对这个案例给出的置信度是「高」，因为两端的提交信息都明确。而对提交信息未说明原因的其他案例，系统会强制写明「提交信息未说明原因，以下为基于改动范围的推断」，并把置信度降为「中」——把判断权交回给人。

6.5 自我校验闭环：审稿人当场质疑了叙事者

这是本次实测中最能说明「三模型分工确实有效」的一条证据。叙事者在前一步使用了「大规模回退」这一表述，而审稿人在拿到代码层校验结果后提出了质疑：

审稿人提出的质疑（原文摘录）

「客观指标显示回滚次数为 0 次，而叙事者将 109fd2c 称为“大规模回退”，这与机械统计的“显式回滚”概念可能存在冲突或过度解读，需确认该提交是 git revert 还是单纯的手动代码清理。」

「假设者将邮件移除与站内通讯引入关联为“方案替代”，但缺乏直接证据证明两者存在因果替换关系，可能只是同期的独立调整。」

这条质疑的价值：它同时暴露了叙事层的措辞过强与假设层的因果跳跃，而这恰恰是单一模型最容易产生的两类偏差（过度概括、虚假因果）。审稿人之所以能抓到第一点，是因为它被明确告知「显式回滚 = 0 次」这一机械统计值——一个只看前两个模型输出的审稿人，无从知道这个矛盾存在。这就是「有依赖的流水线」与「并行投票」的实质差别。

7.1 代码规模与构成（实测统计）

目录	文件数	行数	职责
src/	26	2737	挖掘引擎 + 多模型编排 + 声明式 API 路由 + 重构服务
public/	5	2459	零构建原生前端（无任何框架依赖）
tests/	13	2820	测试代码，规模超过产品代码本身
bin/	2	709	命令行界面（考古 / 重构两条子命令）
scripts/	5	670	本地服务、冒烟测试、案例侦察、构建与产物体检
desktop/	10	—	桌面客户端主进程与安全策略（三平台共用同一线上入口）
合计	61	9395	全部为自有代码

7.2 测试与验证状态

181	0	13	~0.59 s
单元测试通过（本地实跑）	失败 / 跳过 / 待办	测试文件	全套测试耗时

测试覆盖范围包括：挖掘引擎的边界与回归、信噪分类、内容重合率判定的四组真实案例固化、模型传输层语义、API 契约、CLI 写入安全、Web 层 XSS 与凭据清理、以及重构链路的补丁解析、固定 SHA、权限校验、提示注入防护、重复提交与超时阻断。

7.3 安全边界设计

- 凭据不落盘：Web 端 Token 仅存于内存；CLI 仅读取专用环境变量，不读取任何本机站点配置或系统钥匙串。
- 源码内容安检：生成或输出前检测疑似凭据（多形态 token 前缀与私钥头），命中即中止并报错，而不是静默输出。
- 路径与来源收敛：仓库标识、文件路径、Git 地址均做严格白名单校验，拒绝重定向、绝对路径、路径穿越、符号链接与各类生成物。
- 写入最小化：任何远端写操作都需要调用者 Token + 身份 + 权限 + 未变化的基础提交 + 签名审阅凭证 + 人工审阅后的校验值全部匹配；只创建带有 `draft` 标记的独立分支与草稿合并请求，不合并、不写默认分支、失败不自动重试。

一处被主动披露的工程缺陷（值得单独记录）

模型传输层曾长期使用 `redirect`：`'error' —`

这是一个拒绝重定向的安全设计（防止请求头中的密钥被转发到第三方）。Node 接受该取值，Cloudflare 边缘运行时不接受，导致线上模型层实际从未生效。问题未被发现的原因是：本地 CLI 与单元测试都跑在 Node 上，而测试甚至把这个只在 Node 上成立的行为固化成了断言。

修复后改用 `manual` 并手动将 `3xx`

判定为错误，语义等价且边缘可用。其教训被写进了项目约束：测试运行时与生产运行时不一致时，测试覆盖不到该差异。

8.1 一次实现，四种调用形态

形态	适用的人	典型用法
Web 应用	不想装任何东西的人	打开页面输入 owner/name，30 秒内看到完整报告，可导出
HTTP API	要把能力集成进自己系统的人	8 个端点，可单独调用挖掘 / 叙事 / 假设 / 裁决
命令行 CLI	要把检查接进流水线的人	一行命令接入 CI，风险分超阈值以退出码阻断合并
桌面客户端	需要独立窗口与离线提示的场景	macOS / Windows 三平台与网站加载同一线上入口，不复制业务逻辑

四种形态共用同一套引擎、同一套 API 契约、同一套模型层 —— 不存在「演示版」和「真实版」的分叉。

8.2 三个可落地的应用场景

场景一 · 代码交接

人员离职或项目转手时，交接文档通常只写「这个模块做什么」，不写「这里曾经试过什么」。产品输出的是一份风险清单 + 必问问题清单 —— 本次实测中，系统针对 oa-system 自动生成了 3 个交接必问问题，全部锚定在具体提交上（例如「为什么邮件功能上线同一天内就被决定废弃」）。

场景二 · 技术尽调 在有限时间内判断一个代码库的真实健康度。产品提供的不只是分数，而是可复核的证据链：风险分的每一项因子都有原始值，每一处「被放弃的尝试」都有两端 SHA 与内容重合率，尽调方可以逐条回查后自行下结论。

场景三 · 持续监测（CI 闸门）

这是产品从「一次性报告」变成「仪表」的关键。命令行支持指定风险分阈值，超限即以退出码 2 阻断流水线，逼着团队在提交信息烂到无法追溯之前就修正。同一个仓库三个月后再跑一次，即可看到知识可追溯率是否在改善。

8.3 价值的可持续性

一次性报告的价值会随交付而终止；仪表的价值随使用频次增长。产品在设计上刻意把「同一套指标可以被反复测量」作为前提：同样的命令、同样的窗口参数，在任意时间点产生的结果可以纵向比较。这是本产品区别于「跑一次看看」类工具的根本处。

以下每一项都是团队在交付前主动核实并披露的，而非等待他人发现。把它们列在这里，是因为一份只讲优势的价值论证不具备被信任的资格。

边界	具体影响	后续路线
依赖 GitHub REST API	私有仓库与内网仓库读不到完整历史，尽调场景目前限于公开仓库	转为本地 git log：零 API 成本、完整历史、不接触对方凭据
不读取 PR 与 issue	「为什么」目前是带证据的推断，而不是对讨论记录的引用	接入 PR / issue 后，可将推断升级为引用
内容核实 ≠ 语义核实	核实的是「删除的确实是先前新增的行」，不证明「这是一个被放弃的决策」。存在内容对得上、但语义上属于同期重构的个案	当前用「置信度 + 强制提示」缓解：此类案例标注置信度「中」，并写明「提交信息未说明原因，以下为推断」
超大 diff 无法核实	当单次改动过大时上游接口不返回内容，系统拒绝下判断，保留为待核实线索	宁可保留可疑条目，也不丢掉真阳性；不可核实不等于假阳性
风险分是设计选择	五个因子的权重由团队设定，不是行业标准	报告中始终附全部因子原始值，使用者可自行重新加权
桌面端运行验收未完成	三平台安装包已构建且通过完整性校验，但本机启动测试未通过，尚不能断言为环境限制还是兼容性问题；未通过关闭沙箱来掩盖	桌面端定位为「同一线上入口的封装」，不作为独立交付路径；正式发布前需完成真机验收
部分网络环境不可达	部署在同一平台的另一入口（workers 子域）在部分网络环境下被阻断	已切换至实测可达的 Pages 入口；不依赖单一入口是长期方向

9.1 一句话的边界声明

产品不做的事

它不声称能重建决策动机；它不给出「这个项目好不好」的结论；它不在没有证据时下判断。

它只做一件事：把「哪里已经没人能说清为什么」标记出来，并把判断权连同证据一起交回给人。

A · 复现本次报告中的全部线上数据（无需任何凭据）

```
# 健康检查与模型状态
curl -s https://code-archaeology.pages.dev/api/health

# 挖掘一个仓库（纯计算，不调用模型）
curl -s -X POST https://code-archaeology.pages.dev/api/mine \
  -H "content-type: application/json" \
  -d '{"repo":"ody-cai/oa-system","limit":40}'

# 完整三模型流水线（约 30 秒）
curl -s -X POST https://code-archaeology.pages.dev/api/excavate \
  -H "content-type: application/json" \
  -d '{"repo":"ody-cai/oa-system","limit":40}'
```

B · 本地复现（需 Node.js 18+）

```
cd code-archaeology
npm install

# 单元测试：应为 181 通过 / 0 失败
node --test tests/*.test.mjs

# 命令行挖掘
node bin/ca.mjs mine --repo=chalk/chalk --limit=40

# 接入 CI：风险分超过 60 即以退出码 2 失败
node bin/ca.mjs mine --repo=$REPO --fail-on-risk=60

# 本地起服务（浏览器访问 http://127.0.0.1:8787）
node scripts/local-server.mjs 8787
```

C · 数据来源说明

本报告中的全部量化数据来自三个明确来源，均可独立核验：① 线上 `/api/health`、`/api/mine`、`/api/excavate` 三个端点的实时响应（响应时间与状态码一并记录）；② 本地测试套件实跑结果；③ 对项目源码的静态统计。报告未使用任何估算值、历史引用值或占位数据。

产品对外展示的全部仓库数据均来自 GitHub 官方 REST API 的实时拉取，未预置任何数据集，也不存在合成数据出现在演示路径上的情况。这一点已由独立的取证复核报告（针对引用提交的存在性与元数据一致性）另行确认。

编制与签发

编写人	蔡奇均 Qijun Cai · 王一博 Yibo Wang	签发人	蔡奇均 Qijun Cai · 王一博 Yibo Wang
报告时间	2026 年 9 月 20 日 09:59 (UTC+8)	报告性质	产品价值论证 · 内部与评审使用
联系邮箱	cqjody@126.com	线上入口	https://code-archaeology.pages.dev

本报告的全部结论均可由第三方独立复现：按附录命令重取数据，即可得到与正文一致的数值。报告作者欢迎并接受任何逐条核对——这正是本产品所倡导的工作方式：不要求对方相信自己，只要求对方能够检查自己。